

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi
Nakamotosatoshin@
gmx.com
www.bitcoin.org

Translated into Spanish from bitcoin.org/bitcoin.pdf
by Angel León - www.diariobitcoin.com

Abstract. A purely electronic version of cash would allow online payments to be sent directly from one party to another without having to go through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if there is a trusted third party to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network time-stamps transactions by creating a hash of them in a continuous chain of hash-based proof-of-work blocks, forming a ledger that cannot be altered without recreating the proof-of-work. The longest chain serves not only as proof of the sequence of events recorded but also as proof that it originated from the group with the greatest CPU processing power. As long as the majority of CPU processing power is under the control of nodes that do not cooperate to attack the network, these nodes will generate the longest chain and gain the upper hand over attackers. The network itself requires minimal structure. Messages are sent on a best-effort basis, and nodes can leave and rejoin the network as they see fit, accepting the proof-of-work chain of what happened during their absence.

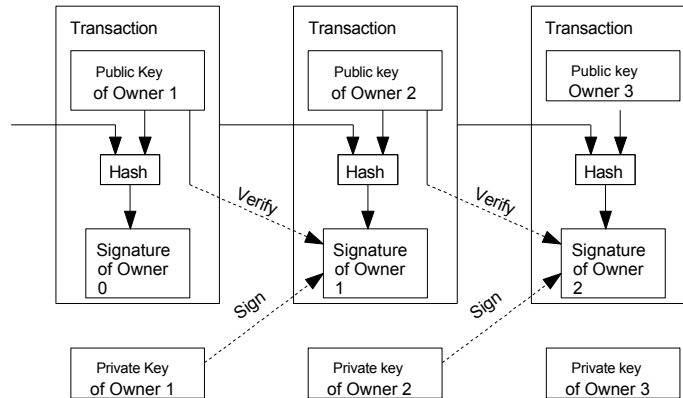
1. Introduction

Internet commerce has come to rely exclusively on financial institutions, which serve as trusted third parties for processing electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust-based model. Completely irreversible transactions are not truly possible, since financial institutions cannot avoid mediating disputes. The cost of mediation increases transaction costs, limiting the practical minimum transaction size and eliminating the possibility of small, casual transactions, and there is a broader cost in the loss of the ability to make non-reversible payments for non-reversible services. With the possibility of reversal, the need for trust expands. Merchants must be cautious with their customers, inconveniencing them by requesting more information than would otherwise be necessary. A certain percentage of fraud is accepted as inevitable. These payment costs and uncertainties can be avoided in person by using physical cash, but there is no mechanism for making payments over a communication channel without a trusted third party.

What is needed is an electronic payment system based on cryptographic proofs rather than trust, allowing two parties interested in conducting transactions to do so directly without the need for a trusted third party. Transactions that are computationally infeasible to reverse would protect sellers from fraud, and routine escrow mechanisms could be easily implemented to protect buyers. In this paper, we propose a solution to the double-spending problem using a distributed peer-to-peer timestamping server to generate a computational proof of the chronological order of transactions. The system is secure as long as honest nodes collectively control more processing power (CPU) than any group of cooperating attacking nodes.

2. Transactions

We define an electronic currency as a chain of digital signatures. Each owner transfers the currency to the next owner by digitally signing a hash of the previous transaction and the next owner's public key, and appending these to the end of the currency. A recipient can verify the signatures to verify the chain of ownership.

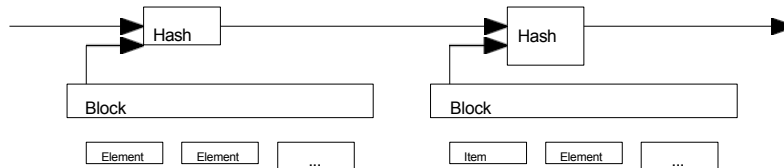


The obvious problem is that the recipient cannot verify whether one of the owners has double-spent the coin. A common solution is to introduce a trusted central authority, or mint, that checks each transaction for double-spending. After each transaction, the coin must be returned to the mint to generate a new coin, and only coins generated directly by the mint are trusted not to have been double-spent. The problem with this solution is that the fate of the entire monetary system depends on the company that manages the mint, with every transaction having to go through them, just like a bank.

We need a way for the recipient to know that the previous owners did not sign any earlier transactions. For our purposes, the earliest transaction is the one that counts, so we don't care about other attempts at double-spending that occur later. The only way to confirm the absence of a transaction is to be aware of all transactions. In the mint model, the mint was aware of all transactions and would decide which ones arrived first. To achieve this without a trusted third party, transactions must be publicly announced [1], and we need a system of participants who agree on a single history of the order in which they were received. The recipient needs proof that, at the time of each transaction, the majority of nodes agreed that it was the first one received.

3. Timestamp Server.

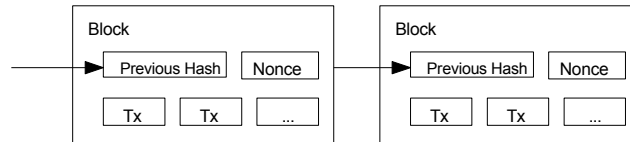
The solution we propose begins with a timestamp server. A timestamp server works by taking a hash of a block of items to be timestamped and widely publishing the hash, such as in a newspaper or a Usenet publication [2–5]. The timestamp proves that the data must have existed at that point in time, obviously, to be included in the hash. Each timestamp includes the previous timestamp in its hash, forming a chain, with each additional timestamp reinforcing those that came before it.



4. Proof-of-Work

To implement a timestamp server on a peer-to-peer basis, we will need to use a proof-of-work system similar to Adam Back's Hashcash [6], rather than a newspaper or a Usenet publication. The proof-of-work involves searching for a value such that, when hashed using an algorithm like SHA-256, the hash begins with a certain number of zero bits. The average work required is exponential in the number of zero bits required and can be verified by computing a single hash.

For our timestamp network, we implement the proof-of-work by incrementing a nonce in the block until a value is found that yields the required number of leading zeros for the block's hash. Once the CPU effort has been expended to satisfy the proof-of-work, the block cannot be altered without redoing all the work. As more blocks are chained after this one, the work required to alter the block would include redoing all the blocks that follow it.



Proof-of-work also solves the problem of determining representation in terms of majority decision-making. If the majority were based on one vote per IP address, it could be subverted by someone capable of assigning many IP addresses. Proof-of-work is essentially “one CPU, one vote.” The majority decision is represented by the longest chain, which has the proof-of-work with the most effort invested in it. If the majority of CPU power is controlled by honest nodes, the honest chain will grow faster and overtake any competing chain. To modify a block in the past, an attacker would have to redo the proof-of-work for that block and all subsequent blocks, and then catch up to and surpass the work done by honest nodes. We will then demonstrate that the probability of a slower attacker catching up decreases exponentially as subsequent blocks are added.

To compensate for increases in hardware speed and the varying interest in running nodes over time, the difficulty of the proof-of-work is determined by a moving average aimed at an average number of blocks per hour. If blocks are generated too quickly, the difficulty increases.

5. The Network

The steps for managing the network are as follows:

- 1) New transactions are broadcast to all nodes.
- 2) Each node collects new transactions into a block.
- 3) Each node works to find a difficult proof-of-work for its block.
- 4) When a node finds a proof-of-work, it broadcasts the block to all nodes.
- 5) The nodes accept the block if all transactions in the block are valid and have not already been spent.
- 6) Nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash.

Nodes always consider the longest chain to be the correct one and begin working to extend it. If two nodes broadcast different versions of the next block simultaneously, some nodes may receive one or the other first. In that case, they work on the one they receive first but keep the other branch in case it becomes longer. The tie is broken when the next proof-of-work is found and one branch becomes longer; the nodes that were working on the other branch then switch to the longer one.

New transaction broadcasts do not necessarily need to reach all nodes. As long as they reach many nodes, they will be included in a block before much time passes. Block broadcasts are also tolerant of lost messages. If a node does not receive a block, it will request it when it receives the next block and realizes that one was missed.

6. Incentive

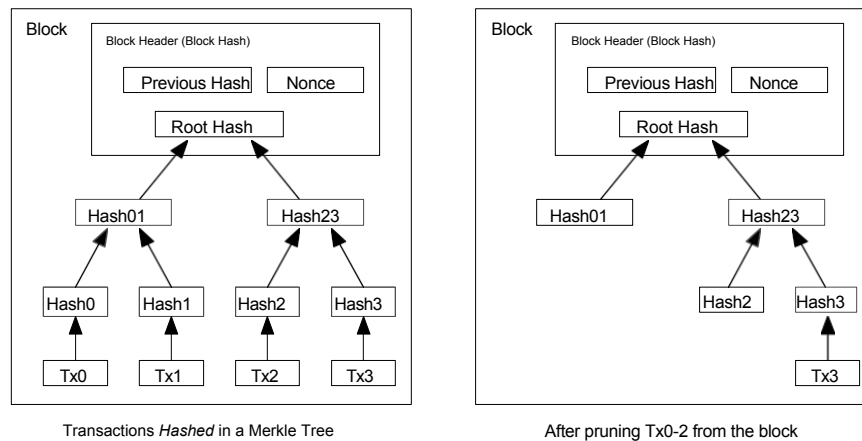
By convention, the first transaction in a block is a special transaction that creates a new coin owned by the block's creator. This provides an incentive for nodes to support the network and offers an initial way to distribute coins into circulation, since there is no central authority to create them. This steady addition of a constant amount of new coins is analogous to gold miners expending resources to add gold to circulation. In our case, it is CPU time and electricity that are expended.

The incentive can also be funded by transaction fees. If the output value of a transaction is less than the input, the difference is a transaction fee that is added to the incentive value of the block containing the transaction. Once a predetermined number of coins have entered circulation, the incentive can transition entirely to transaction fees and become completely inflation-free.

The incentive can help encourage nodes to remain honest. If a self-serving attacker is able to amass more CPU power than all the honest nodes combined, they would have to choose between using it to defraud people by stealing their payments back, or using it to generate new coins. It should be more profitable for them to play by the rules—since those rules favor them with more coins than everyone else combined—rather than to undermine the system and the validity of their own wealth.

7. Reclaiming Disk Space

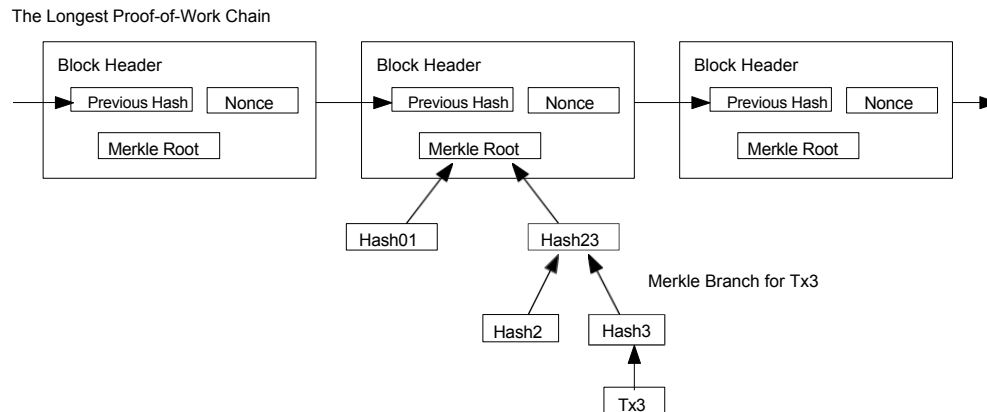
Once the latest transaction in a coin is buried under enough blocks, transactions spent before it can be discarded to save disk space. To facilitate this without breaking the block's hash, transactions are organized into a Merkle tree [7] [2] [5], with only the root included in the block's hash. Old blocks can be compacted by pruning branches from the tree. The internal hashes do not need to be stored.



The header of a block without transactions would be about 80 bytes. If we assume that each block is generated every 10 minutes, $80 \text{ bytes} * 6 * 24 * 365 = 4.2 \text{ MB}$ per year. With computers generally selling with 2GB of RAM by 2008, and Moore's Law predicting the current growth rate of 1.2GB per year, storage should not be a problem even if block headers must remain in memory.

8. Simplified Payment Verification

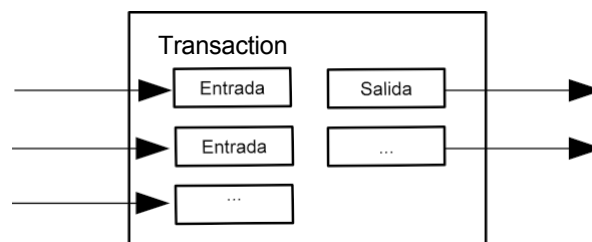
It is possible to verify payments without running a full network node. A user only needs to maintain a copy of the block headers from the longest proof-of-work chain, which can be obtained by searching among network nodes until the user is convinced they have the longest chain, and then obtaining the Merkle branch that links the transaction to the block in which it was timestamped. They cannot verify the transaction on their own, but by linking it to a point in the chain, they can now see that a network node has accepted it, and the blocks added afterward further confirm that the network has accepted it.



As such, verification is reliable as long as honest nodes control the network, but it is more vulnerable if the network is dominated by an attacker. While network nodes can verify transactions on their own, the simplified method can be deceived by an attacker's fabricated transactions until the attacker continues to dominate the network. One strategy to protect against this is to accept alerts from network nodes when they detect an invalid block, prompting the user to download the full block and the flagged transactions to confirm the inconsistency. Businesses that receive frequent payments will want to run their own nodes for more independent security and faster verification.

9. Combining and Splitting Value

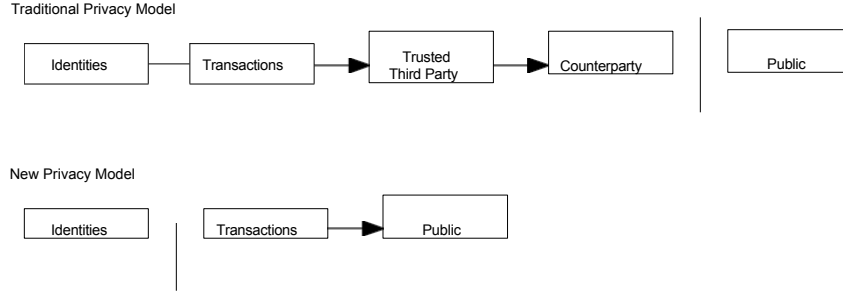
Although it would be possible to manipulate individual coins, it would be cumbersome to make a transaction for every cent in a transfer. To allow value to be split and combined, transactions contain multiple inputs and outputs. Typically, there will be either a single input from a previous, larger transaction or multiple inputs combining smaller amounts, and at least two outputs: one for the payment, and one to return any change, if any, to the sender.



It should be noted that when a transaction depends on several other transactions, and those transactions depend on many more, there is no problem. There is never a need to extract a complete copy of the transaction itself from the transaction history.

10. Privacy

The traditional banking model achieves a level of privacy by limiting access to information to the parties involved and a trusted third party. The need to publicly announce all transactions runs counter to this method, but privacy can still be maintained by breaking the flow of information elsewhere: by keeping public keys anonymous. The public can see that someone is sending an amount to another person, but without any information linking the transaction to any specific individual. This is similar to the level of information disclosed by stock exchanges, where the timing and size of individual transactions—the “tape”—are public, but without revealing the identities of the parties involved.



As an additional safeguard, a new pair of keys must be used for each transaction so that they can be associated with a common owner. Some form of association is inevitable with multi-entry transactions, which may reveal that their entries were owned by the same owner. The risk is that if the owner of a key is revealed, the link could reveal other transactions that belonged to the same owner.

11. Calculations

Consider the scenario in which an attacker attempts to generate an alternative chain faster than the honest chain. Even if this is achieved, it does not open the system to arbitrary changes, such as creating value out of thin air or taking money that never belonged to the attacker. Nodes would not accept an invalid transaction as payment, and honest nodes would never accept a block containing such a transaction. An attacker can only attempt to alter one of their own transactions to reclaim money they have recently spent.

The race between an honest chain and an attacker’s chain can be characterized as a Binomial Random Walk. The success event is the honest chain being extended by one block, increasing its lead by +1, and the failure event is the attacker’s chain being extended by one block, reducing the lead by -1.

The probability that an attacker can catch up from a given deficit is analogous to the Gambler’s Ruin problem. Suppose a gambler with unlimited credit starts at a deficit and plays a potentially infinite number of rounds to try to reach a break-even point. We can calculate the probability that the gambler reaches the break-even point—or that an attacker catches up to the honest chain—as follows [8]:

p = probability that an honest node finds the next block
 q = probability that the attacker finds the next block
 q_z = probability that the attacker catches up starting z blocks back.

$$q_z = \begin{cases} 1 & \text{if } z \leq q \\ (q/p)^z & \text{if } z > q \end{cases}$$

Given our assumption that $p > q$, the probability decreases exponentially as the number of blocks the attacker must reach increases. With the odds stacked against them, if they do not make a lucky guess from the start, their chances become extremely slim as they fall further behind.

Now let's consider how long the recipient of a new transaction needs to wait before having sufficient certainty that the sender cannot alter the transaction. We assume that the sender is an attacker who wants to make the recipient believe that he has paid him for a while, then alter the transaction to pay himself back once some time has passed. The recipient will be alerted when this happens, but the sender hopes it will be too late.

The recipient generates a new key pair and provides the public key to the sender shortly after signing. This prevents the sender from preparing a blockchain ahead of time by working continuously until they are lucky enough to get far enough ahead, and then executing the transaction at that moment. Once the transaction is sent, the dishonest sender begins working secretly on a parallel chain containing an alternate version of their transaction.

The recipient waits until the transaction is added to the block and z blocks have been linked after the transaction. The recipient does not need to know the exact amount of progress the attacker has made, but assuming that the honest blocks took the expected average time per block, the attacker's potential progress will follow a Poisson distribution with an expected value:

$$\lambda = \frac{q}{p}z$$

To obtain the probability that the attacker can still catch up at this point, we multiply the Poisson density for each amount of progress they could have made by the probability that they could catch up from that point:

$$\sum_{k=0}^{\infty} \frac{\lambda^k}{k!} e^{-\lambda} \cdot \begin{cases} (q/p)^{(z-k)} & \text{if } k \leq z \\ 1 & \text{if } k > z \end{cases}$$

We rearrange to avoid summing the infinite tail of the distribution...

$$e^{-\lambda} \sum_{k=0}^z \frac{\lambda^k}{k!} \left(\frac{q}{p} \right)^{(z-k)}$$

Let's convert this to C code...

```
#include <math.h>
double AttackerSuccessProbability(double q, int z)
{
    double p = 1.0 - q;
    double lambda = z * (q / p);
    double sum = 1.0;
    int i, k;
    for (k = 0; k <= z; k++)
    {
        double poisson = exp(-lambda);
        for (i = 1; i <= k; i++)
            poisson *= lambda / i;
        sum -= poisson * (1 - pow(q / p, z - k));
    }
    return sum;
}
```

Let's run some results; we can see that the probability decreases exponentially with z .

```
q=0.1
z=0      P=1.0000000
z=1      P=0.2045873
z=2      P=0.0509779
z=3      P=0.0131722
z=4      P = 0.0034552
z=5      P = 0.0009137
z=6      P = 0.0002428
z=7      P=0.0000647
z=8      P = 0.0000173
z=9      P=0.0000046
z=10     P=0.0000012
```

```
q =
0.3      P=1.0000000
z=0
z=5      P = 0.1773523
z=10     P=0.0416605
z=15     P = 0.0101008
z=20     P = 0.0024804
z=25     P = 0.0006132
z=30     P = 0.0001522
z=35     P = 0.0000379
z=40     P = 0.0000095
z=45     P = 0.0000024
z=50     P = 0.0000006
```

We solve for P less than 0.1%...

```
P < 0.001 q
= 0.10    z =
5
q = 0.15  z = 8
q = 0.20  z=11
q = 0.25  z=15
q=0.30    z=24
q=0.35    z=41
q=0.40    z=89
q = 0.45  z=340
```

12. Conclusion

We have proposed a system for trustless electronic transactions. We start with the standard framework of currencies based on digital signatures, which provides strong ownership control but is incomplete without a way to prevent double-spending. To address this, we have proposed a peer-to-peer network that uses proof-of-work to record a public transaction history, which quickly becomes computationally impractical for an attacker to alter if honest nodes control the majority of CPU power. The network is robust in its unstructured simplicity. Nodes can all operate simultaneously with little coordination. They do not need to be identified, since messages are not routed to any particular destination and need only be delivered on a best-effort basis. Nodes can leave and rejoin the network at will, accepting the proof-of-work chain as evidence of what happened while they were absent. They vote with their CPU power, expressing their acceptance of valid blocks by continuing to work on them and rejecting invalid blocks by refusing to work on them. Any necessary rules and incentives can be enforced through this consensus mechanism.

References

- [1] W. Dai, “b-money,” <http://www.weidai.com/bmoney.txt>, 1998.
- [2] H. Massias, X.S. Avila, and J.-J. Quisquater, “Design of a secure timestamping service with minimal trust requirements,” In *20th Symposium on Information Theory in the Benelux*, May 1999.
- [3] S. Haber, W.S. Stornetta, “How to time-stamp a digital document,” in **Journal of Cryptology**, vol. 3, no. 2, pp. 99–111, 1991.
- [4] D. Bayer, S. Haber, W.S. Stornetta, “Improving the efficiency and reliability of digital time-stamping,” in **Sequences II: Methods in Communication, Security, and Computer Science**, pp. 329–334, 1993.
- [5] S. Haber, W.S. Stornetta, “Secure names for bit-strings,” in *Proceedings of the 4th ACM Conference on Computer and Communications Security*, pp. 28–35, April 1997.
- [6] A. Back, “Hashcash—a denial-of-service countermeasure,” <http://www.hashcash.org/papers/hashcash.pdf>, 2002.
- [7] R.C. Merkle, “Protocols for public key cryptosystems,” in *Proc. 1980 Symposium on Security and Privacy*, IEEE Computer Society, pp. 122–133, April 1980.
- [8] W. Feller, “An Introduction to Probability Theory and Its Applications,” 1957.